CS383 · SOFTWARE ENGINEERING

Lecture 3 Process Models (I)

Midterm summary

Muharib

What's inside

01 What is a Software process model?

02 The Waterfall Model

03 Incremental Development

04 Reuse-oriented Development

05 The four activities: Process activities

06 Activity 1: Specification

07 Activity 2: Development

08 Activity 3: V&V

09 Activity 4: Evolution

10 Coping with change

11 How does it appear in the midterm?

12 Common mistakes

13 Test yourself

01 What is a Software process model?

A **Software process model** is a simplified description of the activities of building software, seen from one particular point of view. Think of it as a road map that the team follows.

It may include: the order of the activities, the inputs and outputs of each activity (**Inputs / Outputs**), and the roles of the people (**Roles**).

Group of processes	Where we explain it
Plan-driven	This lecture
Agile	The next lecture (Lecture 4)

The three generic models

1. The Waterfall Model
2. Incremental Development
3. Reuse-oriented Development

These are generic models (**Generic**). Companies adapt them to create their own model.

02 The Waterfall Model

The phases follow one another like a waterfall. A phase does not start until the one before it

is finished.

```
1. Requirements Definition
```

```
2. System and Software Design
```

```
3. Implementation and Unit Testing
```

```
4. Integration and System Testing
```

```
5. Operation and Maintenance
```

Strengths	Weaknesses
Aligns to system engineering process It fits the way systems engineering works.	**Inflexible to changing requirements** It cannot handle requirements that change.
Complete set of documentation Every phase produces full documents.	**Late discovery of technical problems** Technical problems show up late.
	Sequential: next phase waits for previous phase Each phase has to wait for the one before it.

Appeared in the midterm

The first two Weaknesses appeared word for word as the correct option in an MCQ.

03 Incremental Development

The system is built in increments. Each increment is a version, and the customer sees it and

gives feedback. The three activities run at the same time (**Concurrent activities**):

```
Specification   ⇄   Initial version
```

```
Development     ⇄   Intermediate versions
```

```
Validation      →   Final version
```

Strengths	Weaknesses
Effectively manages evolving requirements It deals well with requirements that keep changing.	**Reduced visibility and control of activities** It is hard to see progress and to control the activities.
Identifies and resolves technical risk early Technical risks are found early.	**May lead to poor structured software** The software structure can get worse after many changes.
Receive early customer feedback The customer's comments arrive early.	

04 Reuse-oriented Development

Its other name is **Component-based**. Instead of building everything from scratch, we put

together ready-made components.

1. Requirements Specification
2. Component Analysis
3. Requirements Modification
4. System Design with Reuse
5. Development and Integration
6. System Validation

Strengths	Weaknesses
Lowers the cost and time of development It saves money and time.	**Compromises made on requirements** We give up some requirements so they fit the ready-made components.
Lowers technical risk There is less technical risk.	**Less control over software evolution** We have less control over how the system changes in the future.

05 The four activities: Process activities

Every model has the same four basic activities:

1. Software specification
2. Software development
3. Software verification and validation
4. Software evolution

Appeared in the midterm

One question asked you to write the four activities in logical order (2 marks).

The models differ in how the activities are ordered: in **Waterfall** they follow one another (Sequential), and in **Incremental** they overlap and repeat (Interleaved).

06 Activity 1: Specification

Its other name is **Requirement engineering**. It means understanding and defining the services required from the system, and the constraints on its operation and development.

Phase	What it produces
Feasibility study	Feasibility report
Requirements elicitation and analysis	System models
Requirements specification	User and system requirements
Requirements validation	Requirements document

07 Activity 2: Development

It has two parts: **Design**, then **Implementation**.

Software design

A description of the structure of the software to be built, such as the data models and the interfaces between components. It is an **Iterative** process: we add details and go back to revise more than once.

Design inputs	Design activities	Design outputs
Platform information Requirements specification Data description	Architectural design Interface design Component design Database design	System architecture Database specification Interface specification Component specification

Software implementation

We take the outputs of the Design and turn them into a working system (**Executable system**).

Approach	How
Manual	The programmer writes the code by hand.
MDD	Models (such as UML) are converted automatically into code or documents.
Agile	The design may exist inside the code itself.

Testing during development

- Errors (**Defects**) can appear at any level: design or implementation.
- Errors spread (**Defects cascade**): a change in one component causes a problem in another component.
- For this reason, testing must be **Iterative**.

08 Activity 3: V&V

Verification and Validation is the process where we check that the system matches its

specification.

#	Testing stage	What we do in it
1	**Component testing**	We test each component on its own.
2	**System testing**	We put the components together and test the whole system.
3	**Acceptance testing**	We test with real data from the customer.

The V-model

In **Plan-driven** processes there are test plans (**Test plans**) that link each development phase to the testing stage that matches it.

Development phase	Test plan it produces
Requirements specification	Acceptance test plan
System specification / System design	System integration test plan
Detailed design	Sub-system integration test plan

09 Activity 4: Evolution

It means changing the system after it is running at the customer's site (**After deployment**).

The three types of maintenance

Type	Meaning	Share
Fault repairs	Fixing errors.	17%
Environmental adaptation	Changes needed because the environment changed: new hardware or a new operating system.	18%
Functional addition	Adding or changing functions because the business or the market changed.	65%

The cost of fixing an error

Type of error	Cost	Reason
Coding error	Cheap	A small change in the code.
Design error	More expensive	More than one component has to be fixed.
Requirement error	Most expensive	The system has to be redesigned.

Steps of the Evolution process

```
Define system requirements  →  Assess existing systems
```

```
Propose system changes      →  Modify systems  →  New system
```

10 Coping with change

Change is certain in any software: requirements change, the market changes, and technologies develop. Change raises the cost because it causes rework (**Rework**).

Two ways to reduce the cost

Approach	Meaning
Change avoidance	We anticipate the change before it happens, before a lot of rework is needed.
Change tolerance	We design the process so that a change can be added at low cost.

Two practical techniques

Technique	What it is	Supports
System prototyping	A trial version with part of the services, used to get the customer's opinion early.	avoidance
Incremental delivery	We deliver the features in increments, and the customer tries them and comments.	avoidance and tolerance

11 How does it appear in the midterm?

I reviewed two midterms. Three questions came from this lecture:

1 MCQ First midterm

A key weakness of the Waterfall model in the slides is:

Answer:

Inflexible to changing requirements and late problem discovery

2 MCQ First midterm

To cope with change, there are two approaches:

Answer:

Prototyping & incremental delivery

3 WRITTEN · 2 MARKS Second midterm

What are the 4 main software engineering activities, in logical order?

Answer:

Specification → Development → Verification and validation → Evolution

The question on choosing the right model for a project (4 marks in each midterm) needs the features of the models from this lecture and from Lecture 4.

Two midterms are a small sample. They show you the style of the questions, but they do not guarantee what the next midterm will contain.

12 Common mistakes

Mistake	Correct
Saying that Waterfall has little documentation	The opposite: its documentation is complete. This is one of its strengths.
Mixing up avoidance and tolerance	**Avoidance**: we anticipate the change beforehand. **Tolerance**: we absorb it at low cost.
Saying that prototyping supports both	**Prototyping** supports avoidance only.
Reversing the order of the testing stages	Component, then System, then Acceptance.
Saying that a Coding error is the most expensive error	The most expensive is **Requirement error**.

13 Test yourself

1. Which model has this feature: **Receive early customer feedback**?
2. Which model has this feature: **Compromises made on requirements**?
3. Which testing stage uses real data from the customer?
4. Which type of error is the most expensive to fix? Why?
5. Which technique supports both avoidance and tolerance?

Answers

#	Answer	Reason
1	**Incremental development**	The customer sees each version and comments on it.
2	**Reuse-oriented**	We give up some requirements so they fit the ready-made components.
3	**Acceptance testing**	It is the last of the testing stages.
4	**Requirement error**	The system has to be redesigned.
5	**Incremental delivery**	Prototyping supports avoidance only.

The Test yourself questions were written by me. They are not from past midterms.